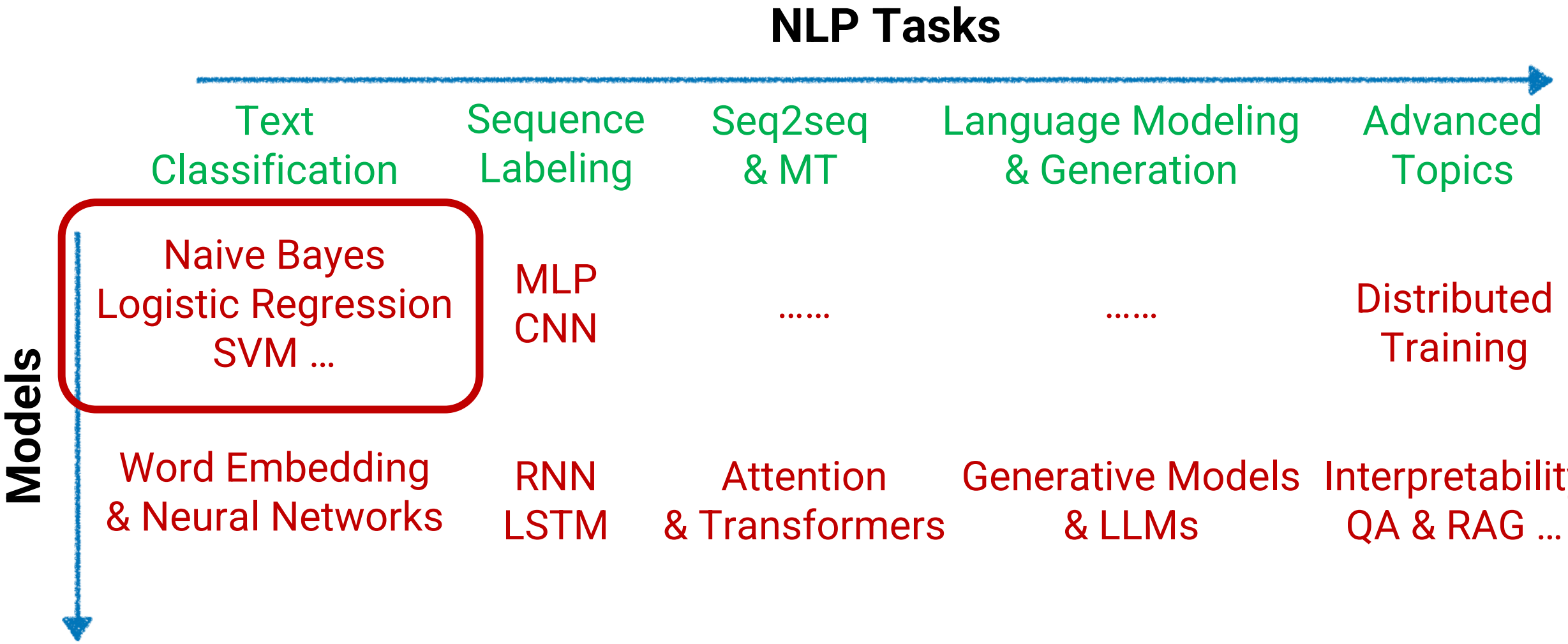
Text Classification Methods: Naïve Bayes, Logistic Regression

Robin Jia & Xuezhe Ma
USC CSCI 544, Spring 2026
January 15, 2026

Announcements

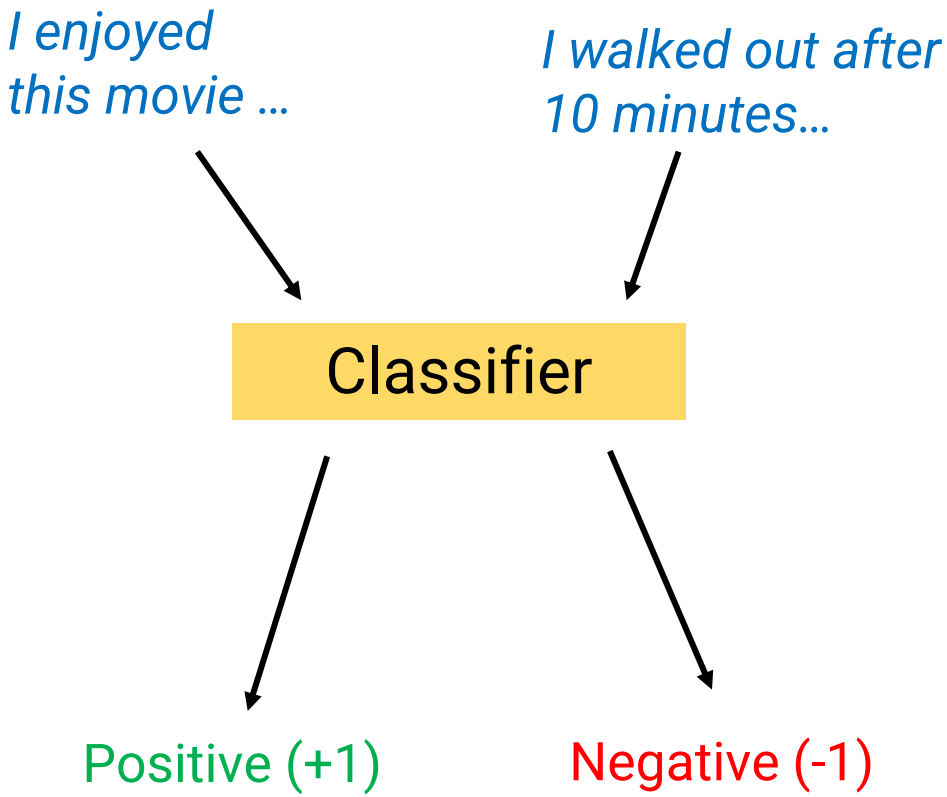
- HW1 will be released today! Due Friday January 30, 11:59pm
- Submit on Gradescope (you all should have been added to the Gradescope course already)
- Office hours google calendar

Course Overview



Text Classification

- Today's goal: Build a system that…
  - Takes in as input a document *x*
  - Predicts whether *x* belongs to one of C classes
- Some text classification problems:
  - Sentiment analysis: Does *x* express positive or negative sentiment?
  - Hate speech detection: Is *x* hate speech?
  - Authorship analysis: Which author (e.g., Shakespeare, Jane Austen, etc.) wrote *x*?



How to Build a Text Classifier?

- Key leverage: Assume we have **training data**
  - Pairs $(x^{(i)}, y^{(i)})$ where i ranges from 1 to N
- Each input *x* is a document
  - Documents can have different numbers of words
- Each training example has corresponding label $y^{(i)}$

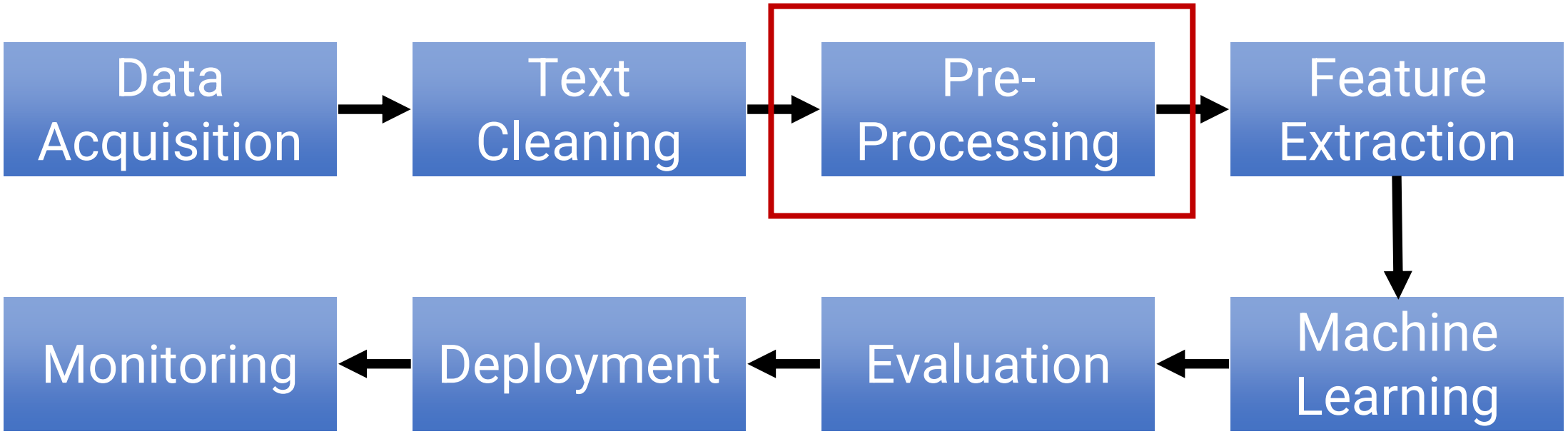
Training Data (sentiment analysis)

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

Today's Plan

- Preprocessing
- Naïve Bayes
  - A "generative" approach
  - Idea: **Count** patterns in the data
- Logistic & Softmax Regression
  - A "discriminative" approach
  - Idea: **Optimize** your predictions based on data
- Model Evaluation

Recall: NLP System Pipeline



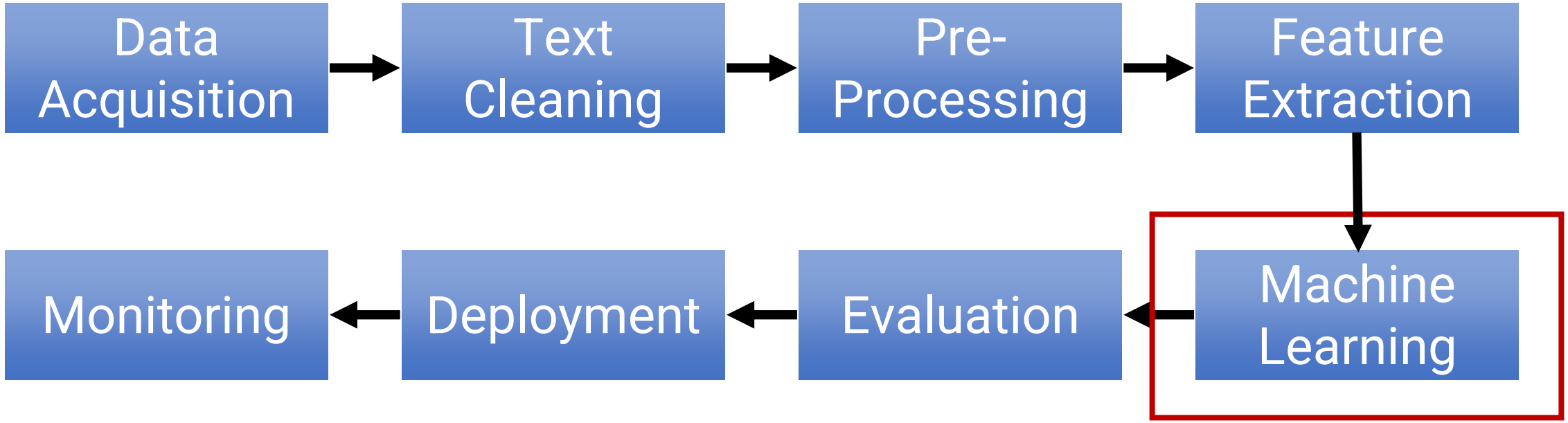
Preprocessing

- Goal: Convert data into a standardized form that our models can easily ingest
- Tokenization: splitting the text into units for processing
  - Removing extra spaces
  - Removing “unhelpful” text (e.g., external URL links)
  - Splitting punctuation: *Happy New Year!* -> *Happy New Year !*

Preprocessing

- Optional operations
  - Contracting and standardizing: *won't* -> *will not*
  - Converting capital letters to lowercase
    - *His aunt comes from Poland* -> *his aunt comes from poland*
  - Removing stopwords (*a*, *the*, *about*, etc.)
  - Stemming or lemmatization (*running* -> *run*; *poorly* -> *poor*)
- Intuition: Standardization helps Generalization
  - If we learn that "*excellent*" is a positive word, then we should generalize this observation to "*Excellent*", "*excellent!*", "*excellently*", etc.
- Trade-off: We lose information ("*I'll be Frank*" vs. "*i'll be frank*")

Recall: NLP System Pipeline



Today's Plan

- Preprocessing
- **Naïve Bayes**
  - A "generative" approach
  - Idea: **Count** patterns in the data
- Logistic & Softmax Regression
  - A "discriminative" approach
  - Idea: **Optimize** your predictions based on data
- Model Evaluation

Generative Classifiers

- **Key idea**: We model two things
  - p(y): For each label y, what is the probability of y occurring?
  - p(x|y): For each label y, what corresponding x's are likely to appear?
- If we have good models of these, **we can predict on a new document *x* using Bayes Rule**:

Prediction of label given input

$$p(y \mid x) = \frac{p(y)p(x \mid y)}{p(x)}$$

Model estimates these

Just for normalization

$$p(x) = \sum_j p(y = j)p(x \mid y = j)$$

Modeling p(y)

- Modeling p(y) is easy: Just count how often each *y* appears!
- Let *C* be the number of possible classes
- Our model learns model parameter $\pi_j = P(y=j)$ for each possible *j*
- Learning: $\pi_j = count(y=j)/n$
  - *count(y=j)*: how often y=j in training data
  - *n*: number of training examples

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

In this dataset: y ∈ {+1, -1} so **C=2**

5 training examples, so **n=5**
y=+1 occurs 3 times, so $\pi_1 = 3/5 = 0.6$
y=-1 occurs 2 times, so $\pi_{-1} = 2/5 = 0.4$

Training a generative classifier

- We have to model two things
  - p(y): For each label y, what is the probability of y occurring?
  - **p(x|y): For each label y, what corresponding x's are likely to appear?**
    - This is much harder because we have to model how documents are generated
    - Today: **Naïve Bayes method**

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

Modeling p(x | y) with Naïve Bayes

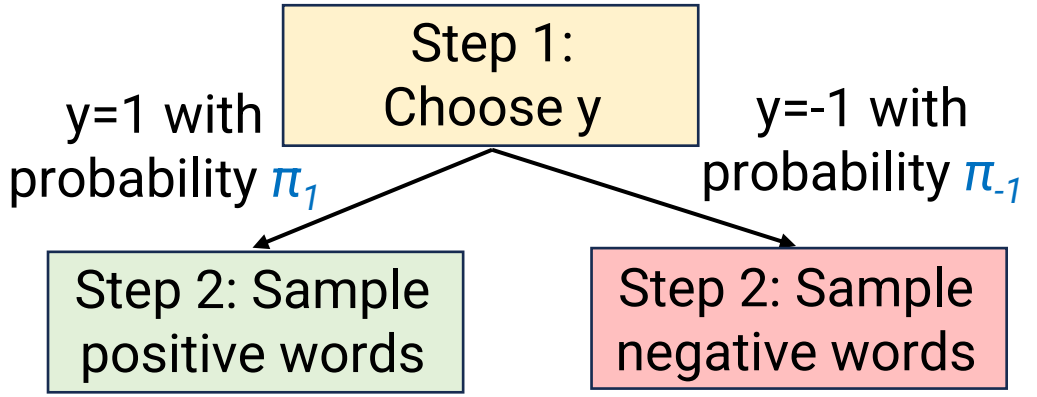
- Idea: Make a simplifying assumption about *p(x|y)* to make it possible to estimate
- Naïve Bayes assumption: Each word of the document *x* is **conditionally independent** given label *y*:

$$p(x \mid y) = \prod_{j=1}^{d} p(x_j \mid y)$$

  - We let x_j denote the *j*-th word of document *x*
  - "Once label is chosen, each word is sampled independently"
  - Note: This assumption does not have to be true (it definitely isn't), just has to be "close enough" so that classifier makes reasonable predictions

The Naïve Bayes Assumption

- Naïve Bayes posits its own **probabilistic story** about how the data was generated
- Step 1: Each $y^{(i)}$ was sampled from the prior distribution $p(y)$
  - "First, decide to either write a positive or negative review"
- Step 2: Each word in $x^{(i)}$ was sampled **independently** from the word distribution for label $y^{(i)}$
  - "If you decided to be positive, write the document by randomly sampling positive-sounding words"
  - "If you decided to be negative, write the document by randomly sampling negative-sounding words"
  - Each word is **independent when conditioning on *y***
- Models the entire process of generating *x* and *y*



P(word|y=1)	word
0.0050	great
0.0042	the
0.0035	good
0.0032	movie
…	…

"movie good the great score…"

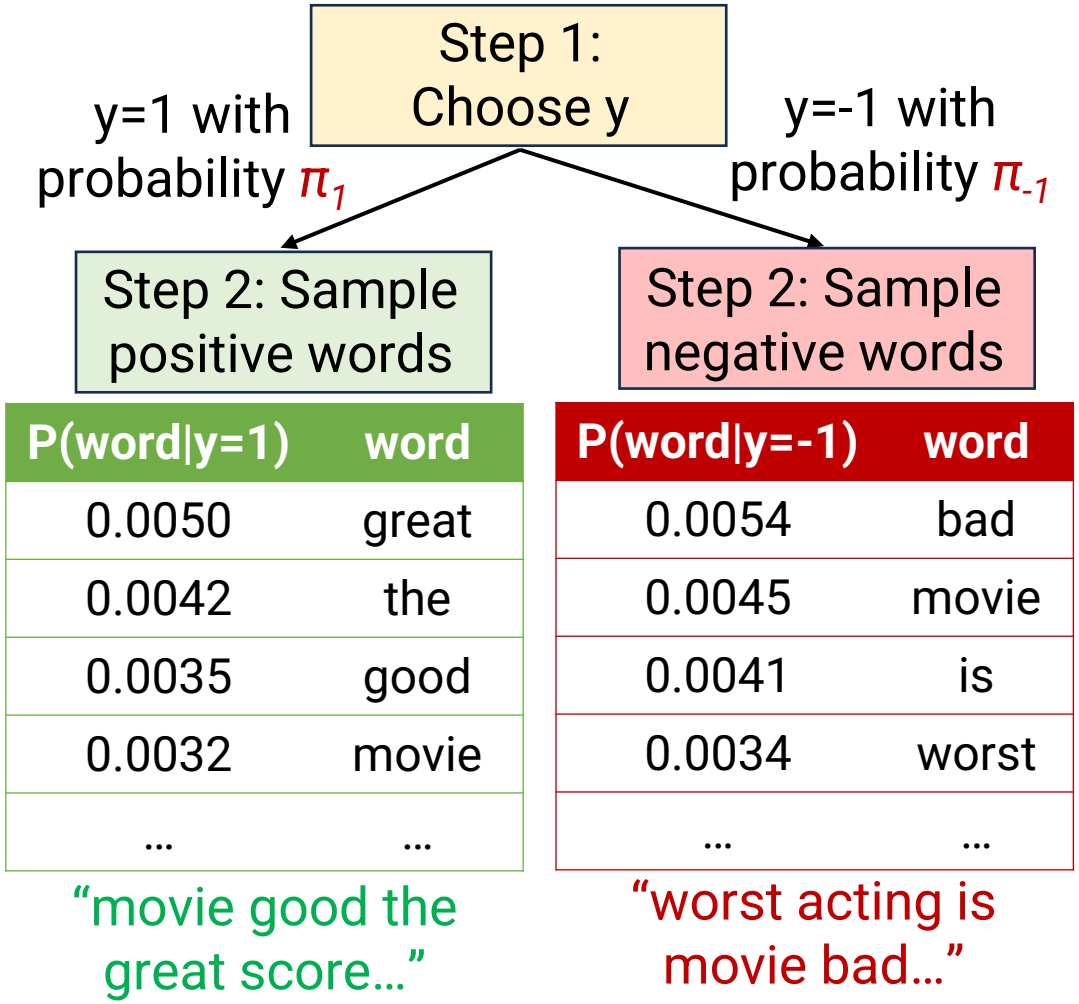
P(word|y=-1)	word
0.0054	bad
0.0045	movie
0.0041	worst
0.0034	is
…	…

"worst acting is movie bad…"

Why is the Naïve Bayes Assumption OK?

- Clearly, documents generated in this way don't look very realistic!
- Why is this OK?
  - We don't need our p(x|y) to actually generate good documents
  - We just need it to be reasonable enough so that when given a real document *x*,

    p(x|true y) > p(x|other y)
  - Can be bad at modeling all the complex things that aren't related to *y* (grammar, writing style, etc.)

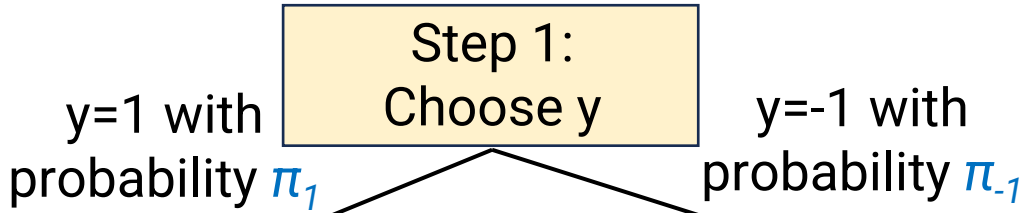


Learning with Naïve Bayes

- Let *V* ("vocabulary") denote the set of words in the dictionary
- Model learns parameter $\tau_{wj} = P(w|y=j)$
  - For each word *w* in *V*
  - For each possible label *j*
  - Total of $|V| * C$ parameters to learn
- How to learn? Just count!
  - For each word w and label j, learn:

$$\tau_{wj} = \frac{[\#occurrences\ of\ w\ when\ y=j]}{[total\ words\ when\ y=j]}$$

  - Note: This formula has a flaw, which we will fix later



Step 2: Sample positive words

P(word|y=1)	word
???	a
???	ab
???	abacus
???	abate
…	…

Step 2: Sample negative words

P(word|y=-1)	word
???	a
???	ab
???	abacus
???	abate
…	…

Learning goal: Estimate all the ???'s

Learning with Naïve Bayes

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

- For each of y=+1 and y=-1, want to learn a distribution over 8 words
- 7 total words appear with y=+1

Parameters to learn

$\tau_{w,1}$	word w
???	acting
???	and
???	amazing
???	directing
???	great
???	movie
???	score
???	terrible

$\tau_{w,-1}$	word w
???	acting
???	and
???	amazing
???	directing
???	great
???	movie
???	score
???	terrible

Learning with Naïve Bayes

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

- For each of y=+1 and y=-1, want to learn a distribution over 8 words
- 7 total words appear with y=+1
- Count each word and divide by total

Parameters to learn

$\tau_{w,1}$	word w
1/7	acting
1/7	and
1/7	amazing
0	directing
2/7	great
1/7	movie
1/7	score
0	terrible

$\tau_{w,-1}$	word w
???	acting
???	and
???	amazing
???	directing
???	great
???	movie
???	score
???	terrible

Learning with Naïve Bayes

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

- For each of y=+1 and y=-1, want to learn a distribution over 8 words
- 7 total words appear with y=+1
- Count each word and divide by total
- Repeat for y=-1 (3 total words)

Parameters to learn

$\tau_{w,1}$	word w
1/7	acting
1/7	and
1/7	amazing
0	directing
2/7	great
1/7	movie
1/7	score
0	terrible

$\tau_{w,-1}$	word w
0	acting
0	and
0	amazing
1/3	directing
0	great
0	movie
0	score
2/3	terrible

Predicting with Naïve Bayes

- Given test example x^{test} = "great score"
- Compute p(x, y=+1)
  = p(y=+1) * p(x|y=+1)
  = p(y=+1) * p("great"|y=+1) * p("score"|y=+1)
  = 3/5 * 2/7 * 1/7 = **0.0245**
- Compute p(x, y=-1)
  = p(y=-1) * p(x|y=-1)
  = p(y=-1) * p("great"|y=-1) * p("score"|y=-1)
  = 2/5 * 0 * 0 = **0**
- By Bayes Rule:
  - P(y=+1|x) = 0.0245/(0.0245+0) = 1
  - Model is sure that y=+1, so predict +1
  - Always predict y with largest p(x, y)

Learned Parameters

$\pi_1 = 3/5$ $\pi_{-1} = 2/5$

$\tau_{w,1}$	word w
1/7	acting
1/7	and
1/7	amazing
0	directing
2/7	**great**
1/7	movie
1/7	**score**
0	terrible

$\tau_{w,-1}$	word w
0	acting
0	and
0	amazing
1/3	directing
0	**great**
0	movie
0	**score**
2/3	terrible

Problem #1: Too Many Zeroes

- Given test example x^{test} = "great directing"
- Compute p(x, y=+1)
  = p(y=+1) * p(x|y=+1)
  = p(y=+1) * p("great"|y=+1) * p("directing"|y=+1)
  = 3/5 * 2/7 * **0** = **0**
- Compute p(x, y=-1)
  = p(y=-1) * p(x|y=-1)
  = p(y=-1) * p("great"|y=-1) * p("directing"|y=-1)
  = 2/5 * **0** * 1/3 = **0**
- By Bayes Rule:
  - P(y=+1|x) = 0/(0+0) = **NaN**
  - **Model thinks this x^{test} is impossible!**

Learned Parameters

$\pi_1 = 3/5$

$\tau_{w,1}$	word w
1/7	acting
1/7	and
1/7	amazing
0	**directing**
2/7	**great**
1/7	movie
1/7	score
0	terrible

$\pi_{-1} = 2/5$

$\tau_{w,-1}$	word w
0	acting
0	and
0	amazing
1/3	**directing**
0	**great**
0	movie
0	score
2/3	terrible

Avoiding Zeroes with Laplace Smoothing

- Problem : Assign probability of 0 to many (word, label) pairs
- Solution: **Laplace Smoothing**
  - Imagine that every (word, label) pair was seem an additional λ times
    - λ is a new hyperparameter
  - New formula:

$$\tau_{wy} = \frac{[\#\text{occurrences of } w \text{ when } y=j] + \boldsymbol{\lambda}}{[\text{total words when } y=j] + \boldsymbol{|V| * \lambda}}$$

Add λ for each word in V,
so total # of imaginary counts is |V| * λ

Parameters to learn

$\tau_{w,1}$	word w
1/7	acting
1/7	and
1/7	amazing
0	directing
2/7	great
1/7	movie
1/7	score
0	terrible

$\tau_{w,-1}$	word w
0	acting
0	and
0	amazing
1/3	directing
0	great
0	movie
0	score
2/3	terrible

Laplace Smoothing Example

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

Parameters to learn

$\tau_{w,1}$	word w
1/7	acting
1/7	and
1/7	amazing
0	directing
2/7	great
1/7	movie
1/7	score
0	terrible

$\tau_{w,-1}$	word w
0	acting
0	and
0	amazing
1/3	directing
0	great
0	movie
0	score
2/3	terrible

With no Laplace Smoothing

Laplace Smoothing Example

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

$$\tau_{wy} = \frac{[\#\text{occurrences of } w \text{ when } y=j] + \boldsymbol{\lambda}}{[\text{total words when } y=j] + |\boldsymbol{V}| * \boldsymbol{\lambda}}$$

Parameters to learn

$\tau_{w,1}$	word w
(1+1)/(7+8)	acting
(1+1)/(7+8)	and
(1+1)/(7+8)	amazing
(0+1)/(7+8)	directing
(2+1)/(7+8)	great
(1+1)/(7+8)	movie
(1+1)/(7+8)	score
(0+1)/(7+8)	terrible

$\tau_{w,-1}$	word w
(0+1)/(3+8)	acting
(0+1)/(3+8)	and
(0+1)/(3+8)	amazing
(1+1)/(3+8)	directing
(0+1)/(3+8)	great
(0+1)/(3+8)	movie
(0+1)/(3+8)	score
(2+1)/(3+8)	terrible

Laplace Smoothing with λ = 1

Laplace Smoothing Example

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

$$\tau_{wy} = \frac{[\#\text{occurrences of } w \text{ when } y=j] + \boldsymbol{\lambda}}{[\text{total words when } y=j] + |\boldsymbol{V}| * \boldsymbol{\lambda}}$$

Parameters to learn

$\tau_{w,1}$	word w
2/15	acting
2/15	and
2/15	amazing
1/15	directing
3/15	great
2/15	movie
2/15	score
1/15	terrible

$\tau_{w,-1}$	word w
1/11	acting
1/11	and
1/11	amazing
2/11	directing
1/11	great
1/11	movie
1/11	score
3/11	terrible

Laplace Smoothing with λ = 1

Laplace Smoothing Avoids Zeroes

- Given test example x^{test} = "great directing"
- Compute p(x, y=+1)
  - = p(y=+1) * p(x|y=+1)
  - = p(y=+1) * p("great"|y=+1) * p("directing"|y=+1)
  - = 3/5 * **3/15** * **1/15** = **0.0080**
- Compute p(x, y=-1)
  - = p(y=-1) * p(x|y=-1)
  - = p(y=-1) * p("great"|y=-1) * p("directing"|y=-1)
  - = 2/5 * **1/11** * **2/11** = **0.0066**
- By Bayes Rule:
  - P(y=+1|x) = 0.0080/(0.0080+0.0066) = **.595**
  - **Model thinks y=+1 is more likely**

Learned Parameters

$\pi_1 = 3/5$ $\pi_{-1} = 2/5$

$\tau_{w,1}$	word w
2/15	acting
2/15	and
2/15	amazing
1/15	**directing**
3/15	**great**
2/15	movie
2/15	score
1/15	terrible

$\tau_{w,-1}$	word w
1/11	acting
1/11	and
1/11	amazing
2/11	**directing**
1/11	**great**
1/11	movie
1/11	score
3/11	terrible

Laplace Smoothing with λ = 1

Problem #2: Numerical Underflow

- Given **long** test example x^{test} = "great directing and acting, amazing score, …"
- Compute *p(x, y=+1)*:
  - *= p(y=+1) * p(x|y=+1)*
  - *= p(y=+1) * p("great"|y=+1) * p("directing"|y=+1) * p("and"|y=+1) * p("acting"|y=+1) *…*
  - If you actually try to do this on a computer, you will get 0!
    - Multiplying many small numbers results in **numerical underflow**
    - Result is so small that it becomes 0

Learned Parameters

$\pi_1 = 3/5$ $\pi_{-1} = 2/5$

$\tau_{w,1}$	word w
2/15	acting
2/15	and
2/15	amazing
1/15	directing
3/15	great
2/15	movie
2/15	score
1/15	terrible

$\tau_{w,-1}$	word w
1/11	acting
1/11	and
1/11	amazing
2/11	directing
1/11	great
1/11	movie
1/11	score
3/11	terrible

Laplace Smoothing with λ = 1

Use Log Space to Avoid Underflow

- Given long test example x^{test} = "great directing and acting, amazing score, …"
- Instead compute ***log p(x, y=+1)***:
  - = ***log*** *p(y=+1)* ***+ log*** *p(x|y=+1)*
  - = ***log*** *p(y=+1)* ***+ log*** *p("great"|y=+1)* ***+ log*** *p("directing"|y=+1)* ***+ log*** *p("and"|y=+1)* ***+ log*** *p("acting"|y=+1)* ***+*** *…*
    - This will not underflow, just adding together some negative numbers
- At test time: compute ***log p(x, y=j)*** for each *j*, choose the *j* with largest value

Learned Parameters

$\pi_1 = 3/5$ $\pi_{-1} = 2/5$

$\tau_{w,1}$	word w
2/15	acting
2/15	and
2/15	amazing
1/15	directing
3/15	great
2/15	movie
2/15	score
1/15	terrible

$\tau_{w,-1}$	word w
1/11	acting
1/11	and
1/11	amazing
2/11	directing
1/11	great
1/11	movie
1/11	score
3/11	terrible

Laplace Smoothing with λ = 1

Summary: Generative Classifiers, Naïve Bayes

- Generative Classifier: Model p(y) and p(x|y)
  - Modeling p(y) is easy (just count how often each label occurs)
  - Modeling p(x|y) is hard
    - Naïve Bayes assumption: Each word/feature of x is **conditionally independent** given y
    - This makes modeling p(x|y) easy: Just count!
    - Need to be careful to avoid zeroes
      - Laplace Smoothing to avoid zero probability of unseen (word, label) pairs
      - Work in log space to avoid numerical underflow
  - Use Bayes Rule to compute prediction p(y|x) from p(y) and p(x|y)

Today's Plan

- Preprocessing
- Naïve Bayes
  - A "generative" approach
  - Idea: **Count** patterns in the data
- **Logistic & Softmax Regression**
  - A "discriminative" approach
  - Idea: **Optimize** your predictions based on data
- Model Evaluation

Logistic Regression

Naïve Bayes (Generative Approach)

- **Count** a bunch of individual events
- Combine our counts in a principled way (Bayes Rule) to make predictions
- **"Generative"**: Models the entire process of generating (x, y)

Logistic Regression (Discriminative Approach)

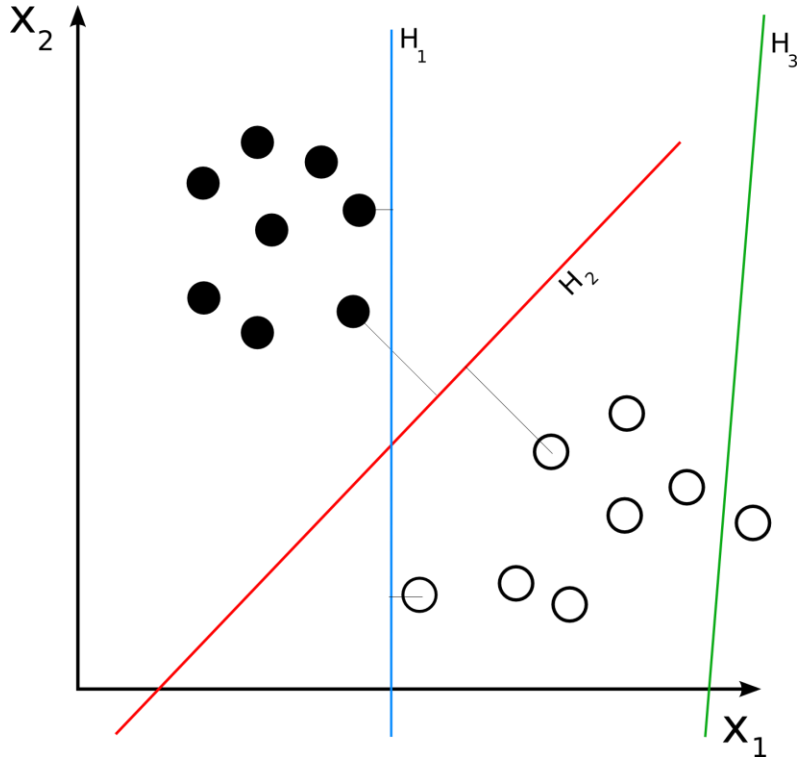
- First decide on a formula we will use to make predictions
  - Formula contains some numerical parameters which determine its output
- **Optimize** the parameters so that we make good predictions on the training data
- **"Discriminative"**: Focuses only on discriminating positives & negatives

Predicting with Logistic Regression

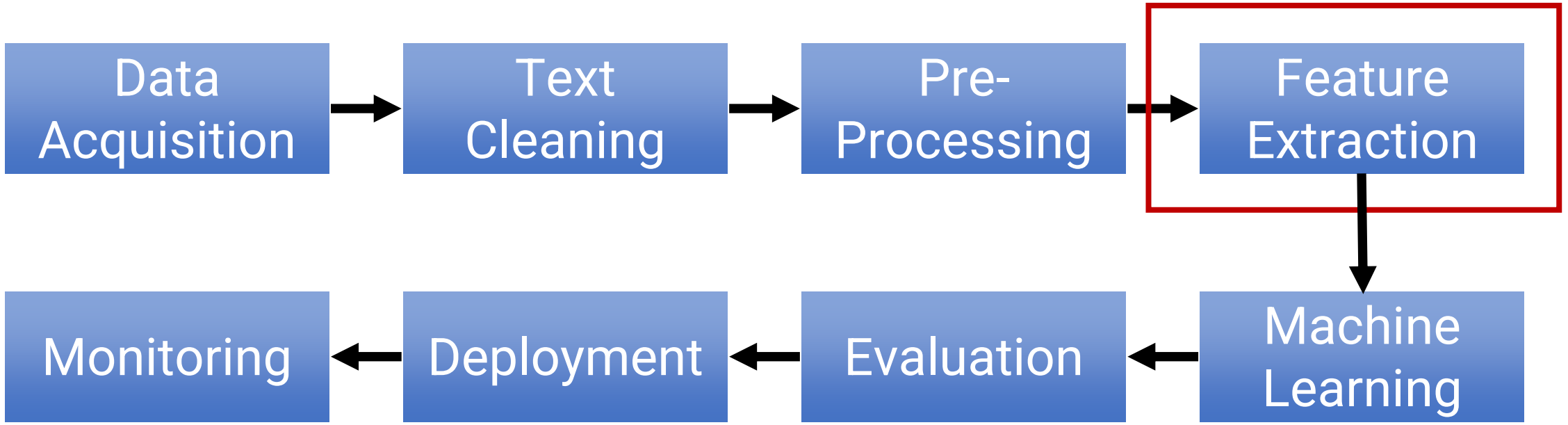
1. Convert the document *x* to a vector of **features** *Φ(x)* of length d
2. Choose (in advance) a **weight vector *w*** of length d and a **bias *b***
   - These are our **parameters**
3. Compute a score as follows:

$$s = \left(\sum_{j=1}^{d} w_j \cdot \phi(x)_j \right) + b = w^\top \phi(x) + b$$

   - Note: This is a *linear model* because score is linear function of features
4. Predict y=+1 if score > 0, otherwise y=-1



Recall: NLP System Pipeline



Concrete Example: Unigram Count Features

- First construct a **vocabulary**: Set of all words in the training data
  - Optional: Can only keep words that occur at most K times, ignore all other words
- **Unigram Count Features**: $\Phi(x)_j$ = # of times j-th vocab word occurs in x

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

Vocabulary *V*

Note: $d = |V| = 8$

index	word
1	acting
2	and
3	amazing
4	directing
5	great
6	movie
7	score
8	terrible

Concrete Example: Unigram Count Features

$\Phi(x)_j$ = # of times j-th vocab word occurs in x

x = "great score and great movie"

Vocabulary *V*
Note: $d = |V| = 8$

index	word
1	acting
2	and
3	amazing
4	directing
5	great
6	movie
7	score
8	terrible

Features Φ(x)

index	value
1	0
2	1
3	0
4	0
5	2
6	1
7	1
8	0

Weight vector w

index	value
1	0.1
2	-0.1
3	5
4	0.4
5	3
6	-0.2
7	0.3
8	-4

Bias *b = 0.1*

Prediction: y=+1

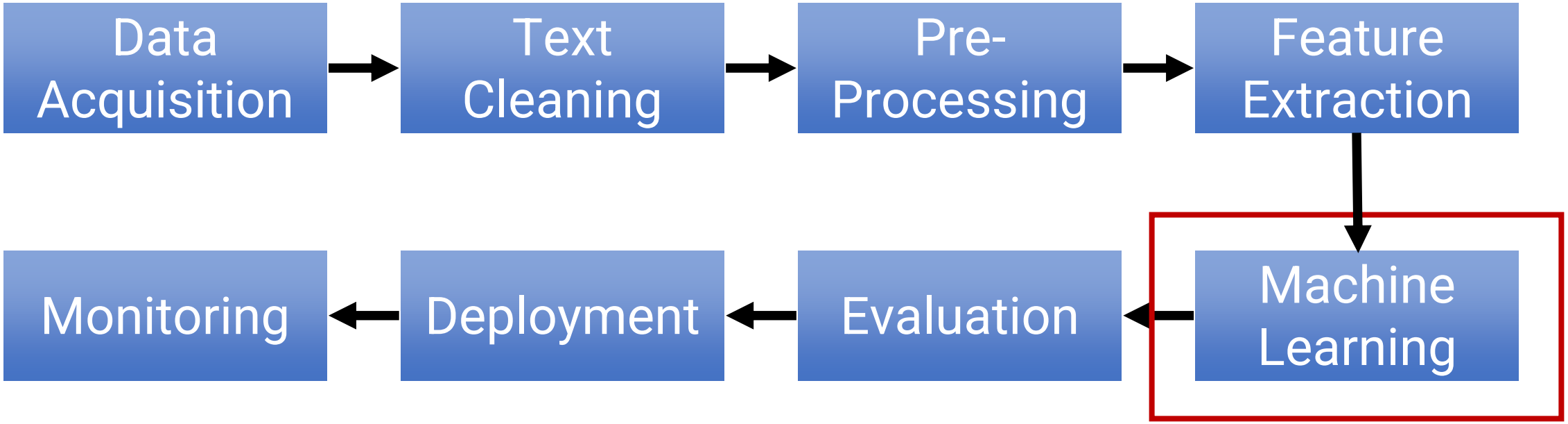
score =

0 * 0.1
\+ 1 * -0.1 ← Subtract 0.1 points for every "and"
\+ 0 * 5
\+ 0 * 0.4
\+ 2 * 3 ← Add 3 points for every "great"
\+ 1 * -0.2
\+ 1 * 0.3
\+ 0 * -4
\+ 0.1 = **6.1** ← Slight bias towards y=+1

Unigram Features: Pros & Cons

- Good: We can learn how each word influences label
- Good: Very simple
- Good: Can work reasonably well for some tasks
- OK: Number of parameters we have to learn is not *that* large
- Bad: We lose all information about order of words
  - We call this a **"Bag of Words"** representation
  - No knowledge of what phrases were said
  - No knowledge of sentence structure
- Bad: Doesn't really capture what the sentence "means"

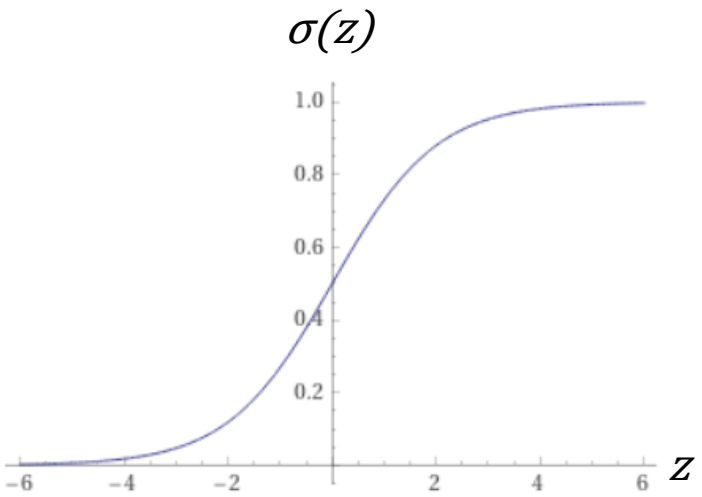
Recall: NLP System Pipeline



How to choose w & b?

- Key idea: Optimize w & b to make sure our predictions are “good” on the training data
- Key question: What does “good” mean?
  - Good: Make correct predictions
  - Better: Make **confidently** correct predictions
    - Score should be very high when y=+1, score should be very low when y=-1
- Approach: Convert scores to probabilities, ask the probability of the right answer to be high

The sigmoid function



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Maps a real number (-∞ to ∞) to a probability (between 0 and 1)
  - 0 gets mapped to 0.5
  - *σ(large number) gets close to 1*
  - *σ(small number) gets close to 0*
- We will treat $\sigma(score) = \sigma(w^T\Phi(x) + b)$ as the **probability that y=+1**, denoted ***p(y=+1 | x; w, b)***
- Consistent with our prediction rule:
  - Recall: We predict y=+1 if score > 0, predict y=-1 otherwise
  - Equivalently: We predict y=+1 if it is the **more likely label**, y=-1 otherwise

Logistic Regression Objective Function

- Now we are ready to write down our objective
  - This is a function of our parameters *w* and *b*
  - We want to *optimize* this objective with respect to w & b
- Intuition: We want the probability of the correct label to be close to 1 on all examples
- Attempt #1: Maximize product of all probabilities

Objective function, depends on w & b

$$\arg\max_{w,b} \prod_{i=1}^{n} p(y = y^{(i)} \mid x^{(i)}; w, b)$$

Find *w* and *b* that maximize this quantity

Probability that our model's prediction equals $y^{(i)}$, the true label, when we feed it document $x^{(i)}$

Logistic Regression Objective Function

Version #1: $\arg\max_{w,b} \prod_{i=1}^{n} p(y = y^{(i)} \mid x^{(i)}; w, b)$

Update: Plug in formula for probability of y

- Fact: $\sigma(y \cdot (w^\top \phi(x) + b))$ is the probability our model assigns to label y
- If y=+1, then this is just $\sigma(w^\top \phi(x) + b) = p(y = +1 \mid x; w, b)$
- If y=-1, then this is $\sigma(-(w^\top \phi(x) + b)) = 1 - \sigma(w^\top \phi(x) + b)$

$$= 1 - p(y = +1 \mid x; w, b)$$

$$= p(y = -1 \mid x; w, b)$$

(By symmetry of sigmoid function)

Version #2: $\arg\max_{w,b} \prod_{i=1}^{n} \sigma(y^{(i)} \cdot (w^\top \phi(x^{(i)}) + b))$

Logistic Regression Objective Function

Version #2: $\arg\max_{w,b} \prod_{i=1}^{n} \sigma(y^{(i)} \cdot (w^\top \phi(x^{(i)}) + b))$

Update: Take log and negate

- We want to convert the product to a sum, which will be easier to deal with
- Log converts multiplication to addition
- Log is a monotonic function, so optimizing F is same as optimizing log F
- By convention, we prefer to minimize a "loss" function rather than maximize, so we negate the objective

Final Version: $\arg\min_{w,b} \sum_{i=1}^{n} -\log \sigma(y^{(i)} \cdot (w^\top \phi(x^{(i)}) + b))$

Find *w* and *b* that **minimize** this quantity

Final objective function/loss function, denoted L(w, b)

How to optimize?

- General strategy to minimize a differentiable function: **Gradient Descent**
- General update rule for any loss function L:

$$w^{(t+1)} = w^{(t)} - \eta \cdot \nabla_w L(w^{(t)}, b^{(t)})$$

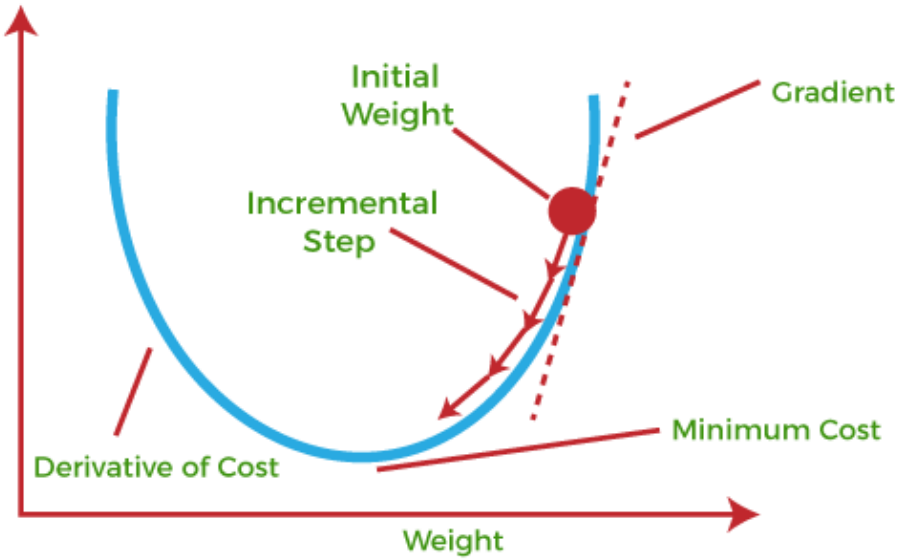
$$b^{(t+1)} = b^{(t)} - \eta \cdot \nabla_b L(w^{(t)}, b^{(t)})$$

Step in direction of negative gradient

New estimate of parameter

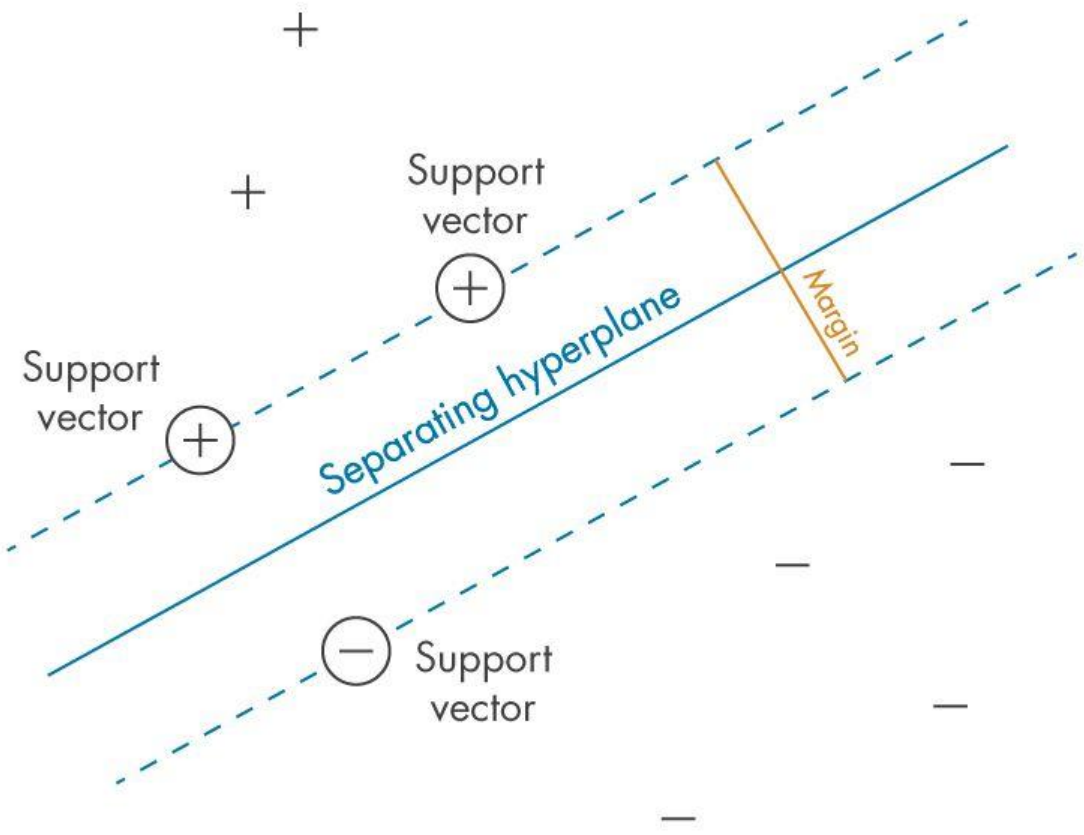
Old estimate of parameter

Learning rate (size of step)



Support Vector Machines

- Another method for training a linear classifier
- Also learns a weight vector *w* and bias *b*
- Key intuition: Find a separating hyperplane with large **margin**
- Location of hyperplane winds up depending on **support vectors** (points closest to boundary)



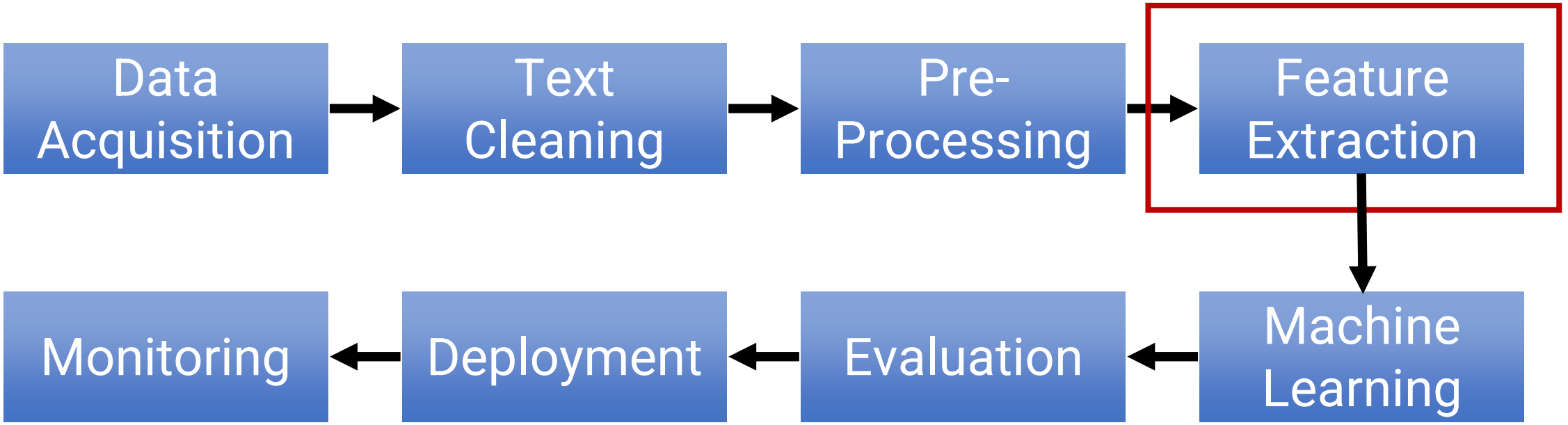
Multi-class/Multinomial Logistic Regression

- What if we have C > 2 classes?
- For each class k = 1, …, C:
  - Learn a separate weight vector w_k
  - Given example x, the score for class k is $w_k^T \Phi(x)$
- Training:
  - The probability of class k is given by a **softmax** function:

$$p(y = k \mid x; w_1, \ldots, w_k) = \frac{\exp(w_k^\top \phi(x))}{\sum_{j=1}^{C} \exp(w_j^\top \phi(x))}$$

  - Minimize negative log probability of data using gradient descent
  - (This is also called softmax regression)

Recall: NLP System Pipeline



Which features to use?

- So far: Unigram features
- Hand-engineered features
  - Look for specific patterns (e.g., with a regular expression)
  - Look for a "type" of word/phrase (e.g., use a list of known "positive" words, have a feature that counts # of positive words)
- Bigram features
  - For each bigram (pair of consecutive words), have a new feature
    - Example: "*great acting and score*" has 3 bigrams: "*great acting*", "*acting and*", "*and score*"
  - Now we have $|V|^2$ possible features
    - Most will = 0 so it's not that expensive to store in memory
  - Now, we learn a separate weight parameter for each bigram count
    - E.g., one weight gives some points for every instance of "*great acting*" in the document
  - Normally we would use unigram **and** bigram features together
- Can also add "n-gram" features for any value of n

Why are bigrams useful?

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	it was good
2	-1	it was not good
3	-1	it was bad
4	+1	it was not bad

- Consider this simple training dataset
  - The unigram "*great*" appears in both positive and negative examples
  - So does "*bad*"
  - So does "*not*"
  - With unigram features alone, can't learn anything from this dataset!
- With unigram + bigram features, we can fit this dataset
  - Strong negative weight (-10) on "not good"
  - Strong positive weight (+10) on "not bad"
  - Weak positive weight (+5) on "good"
  - Weak negative weight (-5) on "bad"

Bigram Features: Pros & Cons

- Good: We can learn how each word **or 2-word phrase** influences label
- Good: Pretty simple
- Good: Can work better than unigrams for some tasks
- Bad: Number of parameters we have to learn is quite large
  - Number of bigrams can be very large (up to $|V|^2$)
  - Easy to overfit
  - Many bigrams may only occur once in the dataset, hard to learn patterns
- Bad: **Still** not that much information about word order
  - Could call this a **"Bag of Bigrams"** representation
  - **Still** no knowledge of sentence structure
- Bad: **Still** doesn't really capture what the sentence "means"

Discriminative vs. Generative Comparison

Logistic/Softmax Regression

- Usually higher accuracy, especially with large dataset
  - P(y|x) usually simpler to learn than P(x|y)
- Easy to add arbitrary features

Naïve Bayes

- Learning is easier—no gradient descent, just count!
- Can be better when dataset is small

Today's Plan

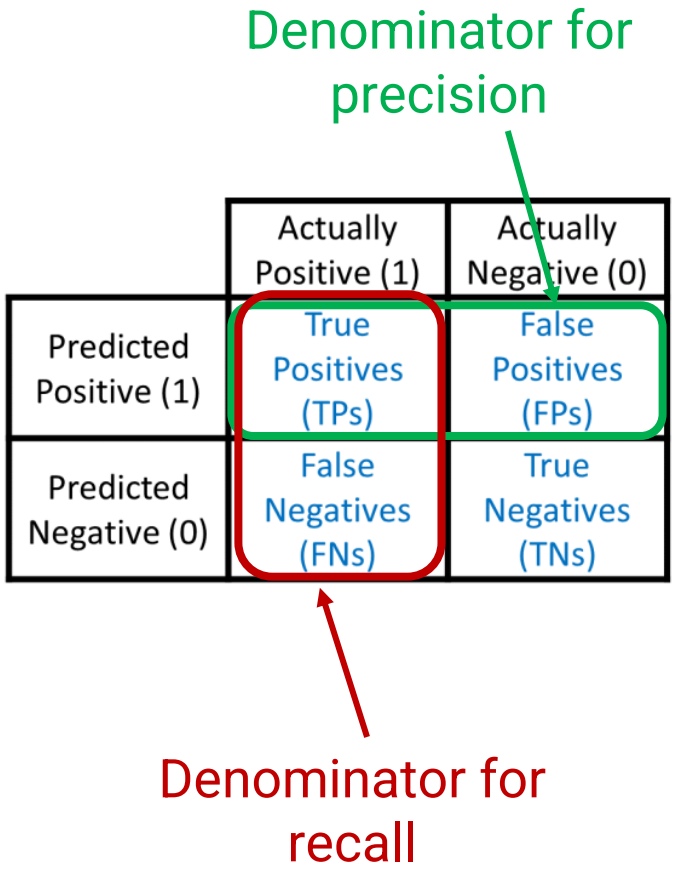
- Preprocessing
- Naïve Bayes
  - A "generative" approach
  - Idea: **Count** patterns in the data
- Logistic & Softmax Regression
  - A "discriminative" approach
  - Idea: **Optimize** your predictions based on data
- **Model Evaluation**

Model Evaluation

- Our goal is always to build a classifier that can classify **any** document
  - Model will overfit to the training data
  - We need to test generalization to **unseen** examples
- Standard practice: Split your dataset into **three parts**
  - **Training set**: Train your model
  - **Development set**: Select hyperparameters (e.g., which features to use, learning rate)
  - **Test set**: Evaluate final model's performance

Evaluation Metrics for Classification

- **Accuracy**: (# correct) / (# total examples)
  - Informative when labels are balanced
  - Not very informative when labels are very imbalanced
  - Hate speech detection: If 99% of data is not hate speech, trivial to get 99% accuracy
- **Precision**: (# true positives) / (# predicted positive)
  - When you predict "positive", how often are you correct? (how *precise* are those predictions?)
- **Recall**: (#true positives) / (# actual positives)
  - Out of all the positives in the dataset, how many did you find? (how many of the positives were *recalled*)
- **F1 score** = (2 * precision * recall) / (precision + recall)
  - Harmonic mean of Precision & Recall
  - More informative when there's label imbalance



Conclusion

- **Preprocessing**: Split text into tokens, optionally do some standardization (depends on what your task is/what information matters)
- **Naïve Bayes**: **Count** word-label co-occurrences + use Bayes Rule to make predictions
- **Logistic & Softmax Regression**: Define features (unigrams, bigrams) & learn weights by **optimizing** for correct predictions
- **Evaluation**: Use held-out data, choose accuracy or F1 depending on label balance